

4.1.1 BNF 定義と CF 文法

¹ 学生実験 c において対象とする原始プログラムを記述するプログラミング言語

```

procedure main(x,y):
  x:=y+1

```

BNF 定義

形式的にはプログラミング言語は文脈自由文法で定義されるが、文脈自由文法では生成規則の集合を記述すれば十分であることが多い。このため、生成規則の集合でプログラミング言語を定義する。その際に一般にBNF(Backus-Nauer Form) 定義と呼ばれる記法が用いられる。BNF は生成規則とほぼ同じであるが、 $\rightarrow$ の代わりに $::=$ を用いる。また、同じ左辺をもつ規則をまとめて、右辺を $|$ で並べて記載する。 $\langle \text{proc_decl} \rangle$ についてのBNF 定義は以下のようになる。

$$\begin{aligned}
 \langle \text{proc_decl} \rangle &::= \text{'procedure' } \langle \text{proc_name} \rangle \text{'(' } \langle \text{id_list} \rangle \text{')' ':' } \langle \text{inblock} \rangle \\
 &\quad | \text{'procedure' } \langle \text{proc_name} \rangle \text{'(' } \langle \text{id_list} \rangle \text{')' ':' } \langle \text{inblock} \rangle \\
 \langle \text{proc_name} \rangle &::= \text{'IDENT'} \\
 \langle \text{id_list} \rangle &::= \text{'IDENT'} \\
 &\quad | \langle \text{id_list} \rangle \text{' , ' } \text{'IDENT'}
 \end{aligned}$$

最左導出と最右導出

導出の途中に現れる系列 $\alpha \in (N \cup \Sigma)^*$ において、 α が複数の非終端記号を含むときどの非終端記号を置き換えて導出するかによって、複数の導出が存在する。例えば、 A, B を非終端記号に含み、 $\{a, b, c\} \subseteq \Sigma$, $A \rightarrow a$, $B \rightarrow b$ という生成規則を持つとき、 ABc に対する導出は $ABc \Rightarrow aBc$ と $ABc \Rightarrow Abc$ がある。 $\alpha \in (N \cup \Sigma)^*$ に対する導出のうち、最も左にある非終端記号を置き換える導出を**最左導出**、最も右にある非終端記号を置き換える導出を**最右導出**といい、それぞれ $\alpha \Rightarrow_{lm} \beta$, $\alpha \Rightarrow_{rm} \beta'$ のように記述する。

α の $A \rightarrow \gamma$ による最左導出 $\alpha \Rightarrow_{lm} \beta$ が存在するとき、 α は $\alpha_1 A \alpha_2$ の形をしていて $\alpha_1 \in \Sigma^*$ である。すなわち、 α の A より左の α_1 は終端記号だけの系列か、 ε であり、 $\beta = \alpha_1 \gamma \alpha_2$ である。

α の $A \rightarrow \gamma$ による最右導出 $\alpha \Rightarrow_{rm} \beta$ が存在するとき、 α は $\alpha_1 A \alpha_2$ の形をしていて $\alpha_2 \in \Sigma^*$ である。すなわち、 α の A より右の α_2 は終端記号だけの系列か、 ε であり、 $\beta = \alpha_1 \gamma \alpha_2$ である。

最左導出、最右導出の0回以上の繰り返しをそれぞれ $\Rightarrow_{lm}^*$, $\Rightarrow_{rm}^*$ で表す。

例 4.2. $PL2$ の $\langle \text{proc_decl} \rangle$ からの最左導出は以下のようになる。

$$\begin{aligned}
 \langle \text{proc_decl} \rangle &\Rightarrow_{lm} \text{'procedure' } \langle \text{proc_name} \rangle \text{'(' } \langle \text{id_list} \rangle \text{')' ':' } \langle \text{inblock} \rangle \\
 &\Rightarrow_{lm} \text{'procedure' 'IDENT' '(' } \langle \text{id_list} \rangle \text{')' ':' } \langle \text{inblock} \rangle \\
 &\Rightarrow_{lm} \text{'procedure' 'IDENT' '(' } \langle \text{id_list} \rangle \text{' , ' 'IDENT' ') ' ':' } \langle \text{inblock} \rangle \\
 &\Rightarrow_{lm} \text{'procedure' 'IDENT' '(' 'IDENT' ',' 'IDENT' ') ' ':' } \langle \text{inblock} \rangle
 \end{aligned}$$

さらに $\langle \text{inblock} \rangle \Rightarrow_{lm}^* \text{'IDENT' ':' 'IDENT' '+' 'NUMBER'}$ であるとき

$$\Rightarrow_{lm}^* \text{'procedure' 'IDENT' '(' 'IDENT' ',' 'IDENT' ') ' ':' 'IDENT' ':' 'IDENT' '+' 'NUMBER'}$$

$PL2$ の $\langle \text{proc_decl} \rangle$ からの最右導出は以下のようになる.

$\langle \text{proc_decl} \rangle \Rightarrow_{rm} \text{'procedure' } \langle \text{proc_name} \rangle \text{'(' } \langle \text{id_list} \rangle \text{')' ':' } \langle \text{inblock} \rangle$
 ここで $\langle \text{inblock} \rangle \Rightarrow_{rm}^* \text{'IDENT' ':'= 'IDENT' '+' 'NUMBER'}$ であるとき
 $\Rightarrow_{rm}^* \text{'procedure' } \langle \text{proc_name} \rangle \text{'(' } \langle \text{id_list} \rangle \text{')' ':' 'IDENT' ':'= 'IDENT' '+' 'NUMBER'}$
 $\Rightarrow_{rm} \text{'procedure' } \langle \text{proc_name} \rangle \text{'(' } \langle \text{id_list} \rangle \text{' , 'IDENT')' ':' 'IDENT' ':'= 'IDENT' '+' 'NUMBER'}$
 $\Rightarrow_{rm} \text{'procedure' } \langle \text{proc_name} \rangle \text{'(' 'IDENT' ',' 'IDENT')' ':' 'IDENT' ':'= 'IDENT' '+' 'NUMBER'}$
 $\Rightarrow_{rm} \text{'procedure' 'IDENT' '(' 'IDENT' ',' 'IDENT')' ':' 'IDENT' ':'= 'IDENT' '+' 'NUMBER'}$

開始記号 S からの 0 回以上の導出によって得られる $\alpha \in (N \cup \Sigma)^*$ (ここで, $S \Rightarrow^* \alpha$) のことを **文形式** (sentential form) という. さらに, $S \Rightarrow_{lm}^* \beta$, $S \Rightarrow_{rm}^a st\gamma$ となる $\beta, \gamma \in (N \cup \Sigma)^*$ のことをそれぞれ左文形式, 右文形式という.

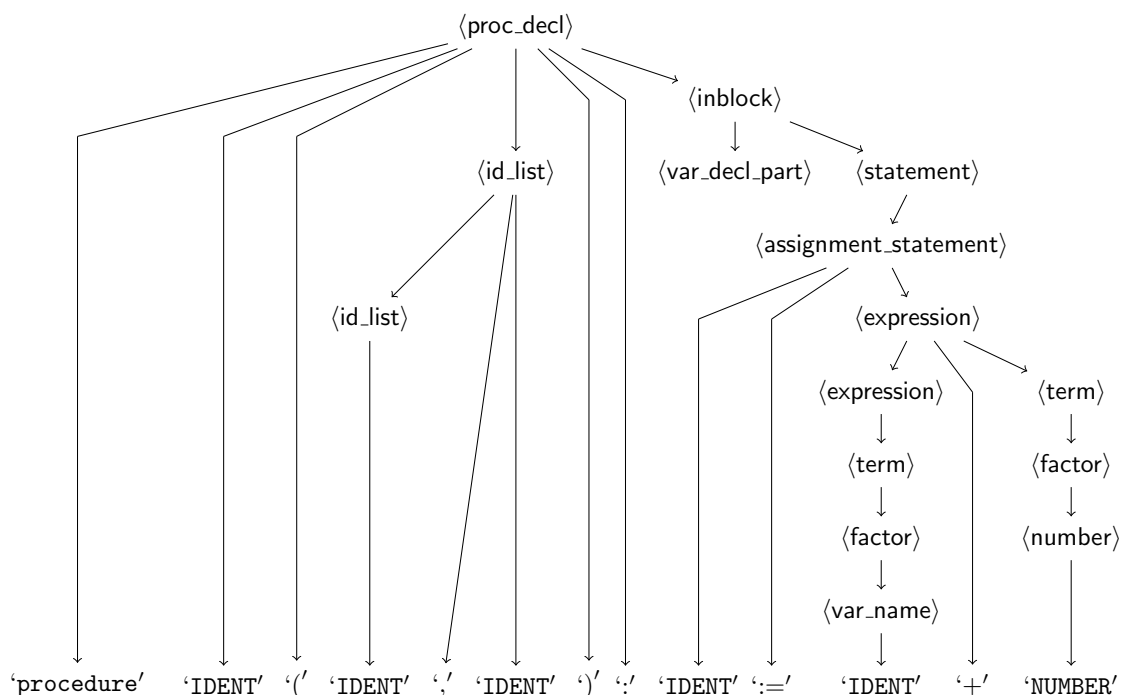
4.1.2 解析木

導出の過程は木構造で表現することができる. 非終端記号, 終端記号を頂点として表し, $A \rightarrow \alpha$ で導出したとき, A の頂点から $\alpha = \alpha_1 \alpha_2 \cdots \alpha_n (\alpha_i \in N \cup \Sigma)$ の α_i に有向辺を追加する. S からの導出をこのようにして表すと, S の頂点を根, $a \in \Sigma$ の頂点を葉とする木構造が構成される. この木構造のことを**解析木** (parse tree)(あるいは**構文木** (syntax tree)) という.

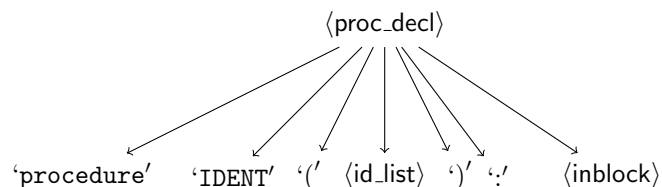
例 4.1 の例の (最左) 導出は以下のようになる.

$\langle \text{proc_decl} \rangle \Rightarrow \text{'procedure' } \langle \text{proc_name} \rangle \text{'(' } \langle \text{id_list} \rangle \text{')' ':' } \langle \text{inblock} \rangle$
 $\Rightarrow \text{'procedure' 'IDENT' '(' } \langle \text{id_list} \rangle \text{')' ':' } \langle \text{inblock} \rangle$
 $\Rightarrow \text{'procedure' 'IDENT' '(' } \langle \text{id_list} \rangle \text{' , 'IDENT')' ':' } \langle \text{inblock} \rangle$
 $\Rightarrow \text{'procedure' 'IDENT' '(' 'IDENT' ',' 'IDENT')' ':' } \langle \text{inblock} \rangle$
 $\Rightarrow \text{'procedure' 'IDENT' '(' 'IDENT' ',' 'IDENT')' ':' } \langle \text{var_decl_part} \rangle \langle \text{statement} \rangle$
 $\Rightarrow \text{'procedure' 'IDENT' '(' 'IDENT' ',' 'IDENT')' ':' } \langle \text{statement} \rangle$
 $\Rightarrow \text{'procedure' 'IDENT' '(' 'IDENT' ',' 'IDENT')' ':' } \langle \text{assignment_statement} \rangle$
 $\Rightarrow \text{'procedure' 'IDENT' '(' 'IDENT' ',' 'IDENT')' ':' 'IDENT' ':'= } \langle \text{expression} \rangle$
 $\Rightarrow \text{'procedure' 'IDENT' '(' 'IDENT' ',' 'IDENT')' ':' 'IDENT' ':'= } \langle \text{expression} \rangle \text{'+' } \langle \text{term} \rangle$
 $\Rightarrow \text{'procedure' 'IDENT' '(' 'IDENT' ',' 'IDENT')' ':' 'IDENT' ':'= } \langle \text{term} \rangle \text{'+' } \langle \text{term} \rangle$
 $\Rightarrow \text{'procedure' 'IDENT' '(' 'IDENT' ',' 'IDENT')' ':' 'IDENT' ':'= } \langle \text{factor} \rangle \text{'+' } \langle \text{term} \rangle$
 $\Rightarrow \text{'procedure' 'IDENT' '(' 'IDENT' ',' 'IDENT')' ':' 'IDENT' ':'= } \langle \text{var_name} \rangle \text{'+' } \langle \text{term} \rangle$
 $\Rightarrow \text{'procedure' 'IDENT' '(' 'IDENT' ',' 'IDENT')' ':' 'IDENT' ':'= 'IDENT' '+' } \langle \text{term} \rangle$
 $\Rightarrow \text{'procedure' 'IDENT' '(' 'IDENT' ',' 'IDENT')' ':' 'IDENT' ':'= 'IDENT' '+' } \langle \text{factor} \rangle$
 $\Rightarrow \text{'procedure' 'IDENT' '(' 'IDENT' ',' 'IDENT')' ':' 'IDENT' ':'= 'IDENT' '+' } \langle \text{number} \rangle$
 $\Rightarrow \text{'procedure' 'IDENT' '(' 'IDENT' ',' 'IDENT')' ':' 'IDENT' ':'= 'IDENT' '+' 'NUMBER'}$

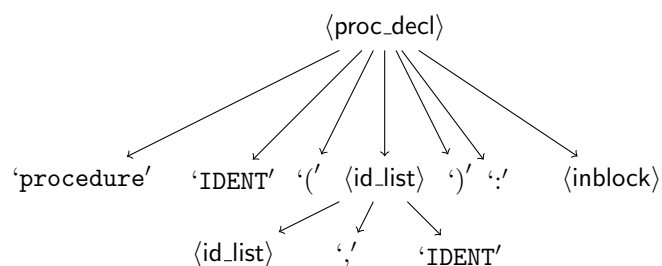
この導出から構成される木構造は以下のようになる.



解析木を構成するにあたって、最左導出では、開始記号は単一の記号なので、必ず最左の非終端記号であり、以下のような木を構成する。

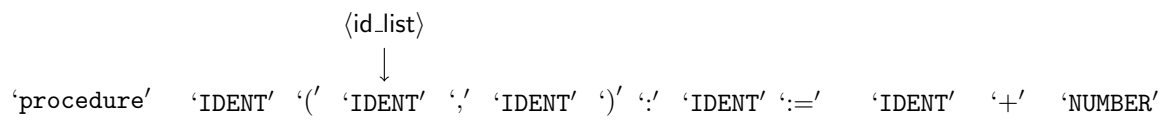


次に最左の非終端記号である <id_list> を導出すると以下の木が得られる。

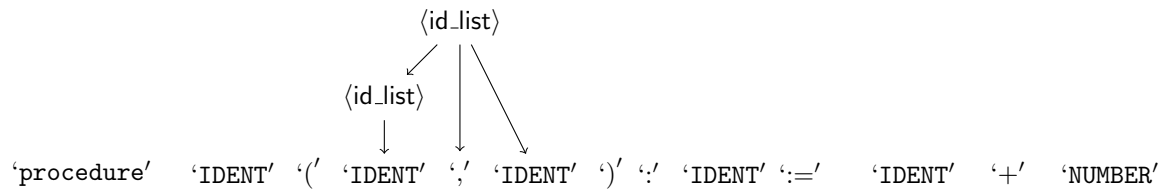


このように最左導出によって構文木は根から下向きに構成される。このような構文解析を**下降型構文解析**という。

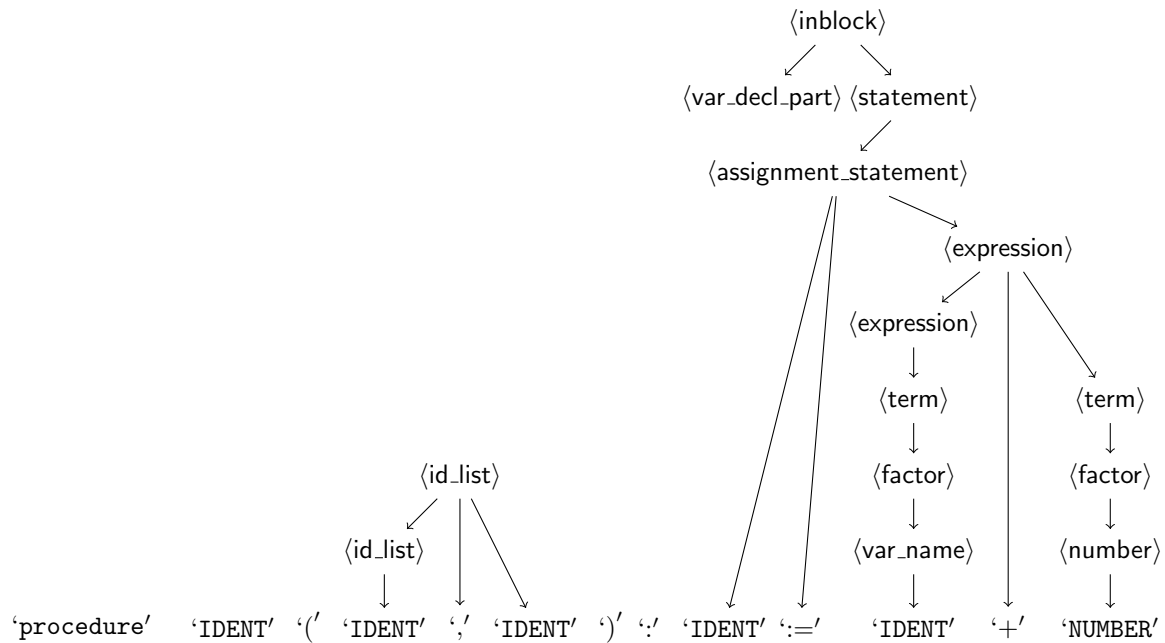
入力系列に対して引数リスト <id_list> を構成する 'IDENT' に <id_list> → 'IDENT' を逆向きに適用し、木を構成する。



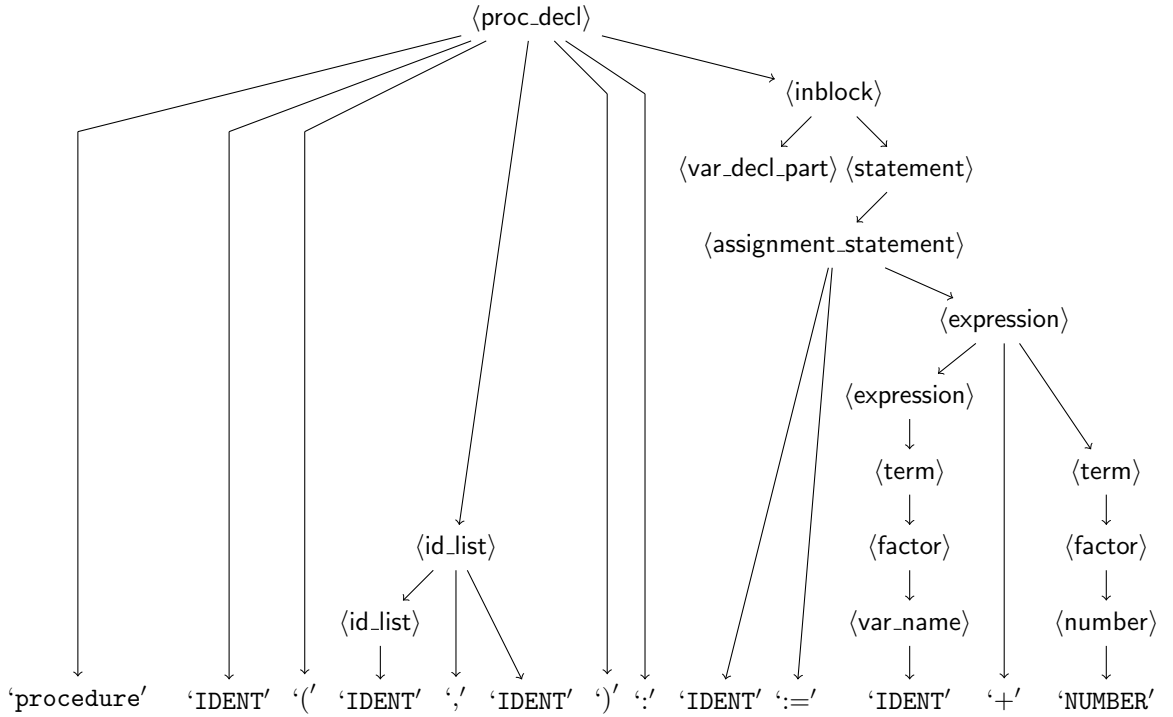
次に, $\langle \text{id_list} \rangle \rightarrow \langle \text{id_list} \rangle \text{'IDENT'}$ を逆向きに適用する.



以下, 右辺にマッチする生成規則を適用し, $\langle \text{inblock} \rangle$ の部分木を生成する.



最後に $\langle \text{proc_decl} \rangle$ を左辺に持つ規則によって木が構成される.



このように構文解析する系列に対して生成規則の右辺をマッチさせて、解析木を構成すると木は葉から根の方向に構成される。このような構文解析を上昇型構文解析という。

4.1.3 文法の曖昧性

<proc_decl> の例においては、どのような順番で生成規則を適用しても構成される解析木は同じである。文法によっては、生成規則の適用の順番によって同じ終端記号系列に対して複数の解析木が構成できることがある。

CF 文法 $G_1 = \langle \{E\}, \{\text{or}, \text{and}, \text{eqt}\}, P_1, E \rangle$ を考える。ここで、

$$P_1 = \left\{ \begin{array}{l} E \rightarrow E \text{ or } E \\ E \rightarrow E \text{ and } E \\ E \rightarrow \text{eqt} \end{array} \right\}$$

とする。eqt は等式のトークンであるとする。

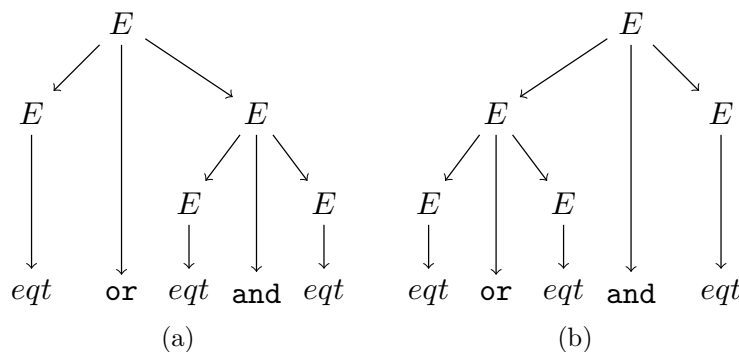


図 4.1: 複数の解析木

$L(G_1)$ に属する $eqt\ or\ eqt\ and\ eqt$ という系列に対して図 4.1 のような 2 つの解析木が存在する。導出において、 $\underline{E}$ を置き換えの対象となった E とすると、(a) は $\underline{E} \Rightarrow \underline{E}\ or\ E \Rightarrow eqt\ or\ \underline{E} \Rightarrow eqt\ or\ \underline{E}\ and\ E \Rightarrow eqt\ or\ eqt\ and\ \underline{E} \Rightarrow eqt\ or\ eqt\ and\ eqt$, (b) は $\underline{E} \Rightarrow \underline{E}\ and\ E \Rightarrow \underline{E}\ or\ E\ and\ E \Rightarrow eqt\ or\ \underline{E}\ and\ E \Rightarrow eqt\ or\ eqt\ and\ \underline{E} \Rightarrow eqt\ or\ eqt\ and\ eqt$, という導出に対する解析木となる。

このように 1 つのトークン系列に対して 2 つ以上の異なる解析木が存在するとき文法は曖昧であるという。

3 つの eqt のトークンが具体的にそれぞれ $x = 1, y = 2, z = 3$ であるとする。 x, y, z がすべて 1 であるとする、 $x = 1$ は真、 $y = 2, z = 3$ は偽となる。 and, or がそれぞれ論理積、論理和を表すとする、(a) の解釈では、最初の $eqt(x = 1)$ が真となるので、全体の論理式は真であるが、(b) の解釈では最後の $eqt(z = 3)$ が偽であるので論理式は偽となる。

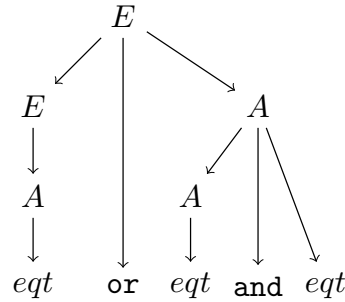
曖昧な文法では入力トークンの系列の意味が定まらないため、プログラミング言語の文法は曖昧でないことが望まれる。

論理和と論理積の場合、一般には和が積が優先されると解釈されるため、(a) の解釈をとる。これにそって G_1 の曖昧さを除去した文法 G_2 を以下に示す。

$$G_2 = \langle \{E, A\}, \{or, and, eqt\}, P_2, E \rangle$$

$$\text{ここで, } P_2 = \left\{ \begin{array}{l} E \rightarrow E\ or\ A \\ E \rightarrow A \\ A \rightarrow A\ and\ eqt \\ A \rightarrow eqt \end{array} \right\}$$

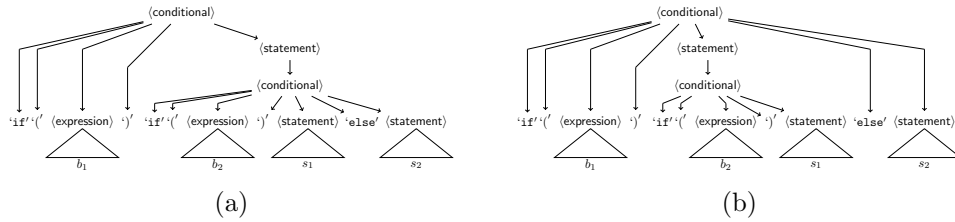
$L(G_1) = L(G_2)$ であり、 G_2 は曖昧ではない。これは、 and を or より優先するために、新たな非終端記号 A を導入して、 and が解析木で or よりも葉に近いように解釈されるようにしたためである。 G_2 では、単一生成規則 (single production) “ $E \rightarrow A$ ” を導入したため解析木のサイズが増大する。 n レベルの優先度の導入のために n 個の非終端記号が導入され、 n 個の単一生成規則が必要になる。曖昧性を除去するトレードオフとして、解析木が大きくなり、解析により長い時間がかかることになる。



このため、生成規則の適用順序を決めて解析の曖昧性を除去する場合もある。C 言語などの条件分岐は `else` 付きの場合と `if ~ then ~ else ~`, `else` なしの場合 `if ~ then ~` の場合がある。

$$\begin{aligned} \langle \text{statement} \rangle &::= \langle \text{conditional} \rangle \mid \dots \\ \langle \text{conditional} \rangle &::= \text{'if' '(' } \langle \text{expression} \rangle \text{' ' } \langle \text{statement} \rangle \\ &\quad \mid \text{'if' '(' } \langle \text{expression} \rangle \text{' ' } \langle \text{statement} \rangle \text{' else' } \langle \text{statement} \rangle \end{aligned}$$

b_1, b_2 を $\langle \text{expression} \rangle$ から導出されるトークン列, s_1, s_2 を $\langle \text{statement} \rangle$ から導出されるトークン列とする。if (b_1) if (b_2) s_1 else s_2 に対して図??のような2つの解析木が存在する。



C 言語では、`else` は最も近い条件に対するものであると解釈するので、解析木として (b) であると決め、曖昧さを除去している。したがって、 b_1 が真で、 b_2 が偽であると評価された場合に実行される。構文解析における曖昧性は望ましいものとは言えないので、プログラム言語を定義する場合は無曖昧な CF 文法で定義することを基本とする。

上記の場合は、新たな予約語 `fi` を導入することで、曖昧さを解消できる。

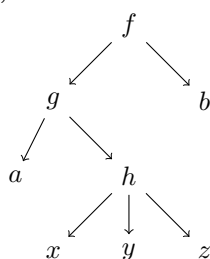
$$\begin{aligned} \langle \text{statement} \rangle &::= \langle \text{conditional} \rangle \mid \dots \\ \langle \text{conditional} \rangle &::= \text{'if' '(' } \langle \text{expression} \rangle \text{' ' } \langle \text{statement} \rangle \text{' fi' } \\ &\quad \mid \text{'if' '(' } \langle \text{expression} \rangle \text{' ' } \langle \text{statement} \rangle \text{' else' } \langle \text{statement} \rangle \text{' fi' } \end{aligned}$$

4.1.4 解析木の表現

木の頂点の出力次数がわかっているとき解析木をトラバースする方法に基づいて、木構造を1次元的に表現することができる。木 T の根頂点を $root(T)$ と書くことにする。 $root(T)$ の次数が n であるとき $root(T)$ からの子を根頂点とする木を左から $T_1, T_2, \dots, T_n$ とする。

- **プリオーダー** $root(T)$ の次に T_1 のプリオーダー系列 $pre(T_1)$ を連結し、その次に $pre(T_2)$ を連結し、というのを再帰的に T_n まで繰り返す.
- **インオーダー** T_1 のインオーダーの系列 $in(T_1)$ に $root(T)$ を連結し、その次に $in(T_2), \dots, in(T_n)$ を連結する.
- **ポストオーダー** T_1 のポストオーダー $post(T_1), \dots, post(T_n)$ を連結し、最後に $root(T)$ を連結する.

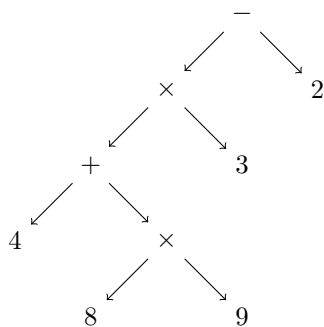
関数を表す式 $f(g(a, h(x, y, z)), b)$ に対して、関数名とその引数を親子関係として木構造で表すと、以下ようになる.



この木構造はプリオーダーで表すと $fgahxyzb$ となる.

f が 2 引数, g が 2 引数, h が 3 引数であることが既知であれば、この系列から関数の形を再現できる.

四則演算の算術式 $(4 + 8 \times 9) \times 3 - 2$ は以下のような構造を持つ.



これをポストオーダー系列で表すと $489 \times +3 \times 2-$ となる.

この記法は**逆ポーランド記法**と呼ばれる. それぞれの演算の引数が決まっていればスタックに対するルールに従って計算すれば式の評価が行われる.

逆ポーランド記法における評価の方法は以下のとおりである.

1. 数字がきたらスタックに積む
2. 演算がきたら引数の数だけスタックからポップして演算を施し、その結果をスタックにプッシュする.
3. 入力を全部読んだあとにスタックトップのあるのが評価値

$489 \times +3 \times 2-$ をスタックと対にしてこの方法で評価すると以下ようになる.

$$\begin{aligned}
(489 \times +3 \times 2-, []) &\rightarrow (89 \times +3 \times 2-, [4]) \\
&\rightarrow (9 \times +3 \times 2-, [4, 8]) \\
&\rightarrow (\times +3 \times 2-, [4, 8, 9]) \\
&\rightarrow (+3 \times 2-, [4, 72]) \\
&\rightarrow (3 \times 2-, [76]) \\
&\rightarrow (\times 2-, [76, 3]) \\
&\rightarrow (2-, [228]) \\
&\rightarrow (-, [228, 2]) \\
&\rightarrow (\epsilon, [226])
\end{aligned}$$

プリオーダーは、木の根を開始記号とする最左導出の規則適用に相当し、ポストオーダーは、木の根を開始記号とする最右導出の逆順の規則適用に相当する。

4.1.5 First と Follow

入力系列 w が与えられたとき、できるだけ効率的に CF 文法に従って $w \in L(G)$ (あるいは $w \notin L(G)$) を示したい。すなわち、 $S \Rightarrow^* w$ となる導出を示す必要がある。この際に導出に使う生成規則を選択する必要がある。例えば、 $A \rightarrow aB, A \rightarrow bC$ という2つの規則があり、 $\beta A \gamma$ という文形式から次の導出を行う。このとき、両方の生成規則が適用可能である。簡単のために最左導出であると考え、 $\beta \in \Sigma^*$ であり、入力文字列 w の接頭語が βa であるならば $A \rightarrow aB$ を選択すべきであり、接頭語が βb であるならば、 $A \rightarrow bC$ を選択すべきである。

このように次に処理すべきトークン (上の場合は β の次のトークン) によって生成規則の選択を決めることができる場合がある。

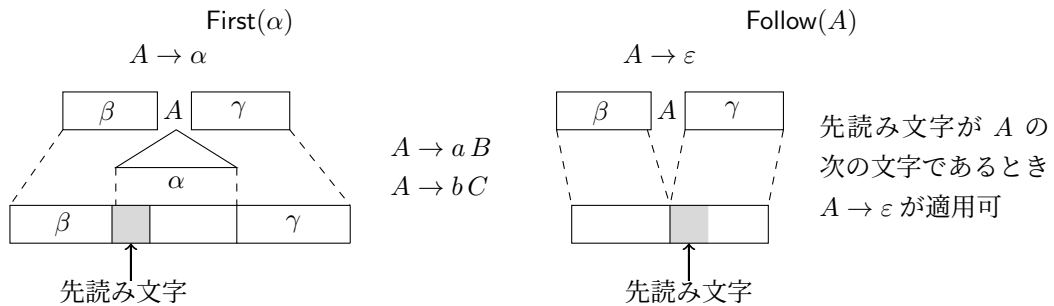


図 4.2: 導出の際の先読みによる決定

さらに、 $A \rightarrow \epsilon$ の場合は、 A に続く文字が何かによってこの生成規則を適用できるかどうかを決めることができる場合がある。

コンパイラの構文解析においては、このようなまだ読み込んでいない**先読み文字**の情報をを用いて、生成規則の選択を決めることが行われる。もしこのような先読み文字の情報をを用いない場合は、すべての選択を試してみることになる。生成規則の選択がまちがっていた場合は、まちがいをやりなおす**バックトラック**によって規則を選び直さないといけなくなる。 βb が接頭語であるに

もかわらず、 $A \rightarrow aB$ で書き換えた場合は、書き換えをやりなおして、 $A \rightarrow bC$ に書き換える。このとき一旦読んでしまったトークンをまた入力系列に戻す。このような操作には時間がかかるため、一般的なプログラミング言語の設計においては生成規則の解析と先読み文字によって、適用できる生成規則が一意的になるような CF 文法で設計される。

生成規則が適用されるときに生成される接頭語を First、生成規則が適用されて生成される文字列の直後の接頭語を Follow といい、これらは生成規則をあらかじめ解析することによって得られる。

定義 4.2. CF 文法 $G = \langle N, \Sigma, P, S \rangle$ が与えられたとき、系列 $\alpha \in (N \cup \Sigma)^*$ に対する Follow_1 集合を以下のように定義する。

$$\alpha \Rightarrow^* \varepsilon \text{ のとき, } \text{First}_1(\alpha) = \{a \in \Sigma \mid \alpha \Rightarrow^* a\beta, \beta \in (N \cup \Sigma)^*\} \cup \{\varepsilon\}$$

$$\alpha \not\Rightarrow^* \varepsilon \text{ のとき, } \text{First}_1(\alpha) = \{a \in \Sigma \mid \alpha \Rightarrow^* a\beta, \beta \in (N \cup \Sigma)^*\}$$

$\text{First}_1(\alpha)$ は、 α から導出によって生成される記号列で先頭に出現する可能性のある終端記号の集合を表す。 α から ε が生成される、つまり、 α が導出によって消失しうる場合には、 $\text{First}_1(\alpha)$ には ε という特別な要素を含めることにする。明らかに $\text{First}_1(\varepsilon) = \{\varepsilon\}$ である。

$\alpha \in (N \cup \Sigma)^+$ に対して、 α の最初の記号を $\text{fst}(\alpha)$ 、最初の記号を除いた系列を $\text{fol}(\alpha)$ と書くことにする。 $(\text{fst}(\alpha) \in N \cup \Sigma, \text{fol}(\alpha) \in (N \cup \Sigma)^* \text{ となる。})$ CF 文法 $\langle N, \Sigma, P, S \rangle$ において、 PO_P を以下のように定める。 $PO_P = \{\gamma \in (N \cup \Sigma)^* \mid \beta\gamma = \alpha, \beta \in (N \cup \Sigma)^*, A \rightarrow \alpha \in P\}$ PO_P は、生成規則 $A \rightarrow \alpha$ の右辺 α の接尾語の集合を表す。

$\alpha \in PO_P \cup N$ に対して $\text{First}_1(\alpha)$ は、以下のようにして計算することができる。

アルゴリズム 4.1. (First_1 を求めるアルゴリズム):

ステップ 1: $\text{First}_1(\varepsilon) \leftarrow \{\varepsilon\}$

ステップ 2: $\alpha \in PO_P \setminus \{\varepsilon\}$ に対して

(a) $\text{fst}(\alpha) \in \Sigma$ ならば、 $\text{First}_1(\alpha) \leftarrow \{\text{fst}(\alpha)\}$

(b) $\text{fst}(\alpha) \in N$ ならば、 $\text{First}_1(\alpha) \leftarrow \emptyset$

ステップ 3: すべての $\text{First}_1(\alpha)$ ($\alpha \in PO_P \cup N$) に新たな要素が追加されなくなるまで以下を繰り返す。

(a) $\text{fst}(\alpha) = A \in N$ である α に対して $\text{First}_1(\alpha)$ に以下の要素を追加する。

$\varepsilon \notin \text{First}_1(A)$ のとき、 $\text{First}_1(A)$

$\varepsilon \in \text{First}_1(A)$ のとき、 $\text{First}_1(A) \setminus \{\varepsilon\} \cup \text{First}_1(\text{fol}(\alpha))$

(b) $A \rightarrow \beta \in P$ に対して $\text{First}_1(\beta)$

定義 4.3. CF 文法 $G = \langle N, \Sigma, P, S \rangle$ に対して、 $A \in N$ に対する $\text{Follow}_1(A)$ は以下のように定義される。

$$\begin{aligned} \text{Follow}_1(A) = & \{a \in \Sigma \mid S \Rightarrow^* \beta A a \gamma, \beta, \gamma \in (N \cup \Sigma)^*\} \\ & \cup \{\# \mid S \Rightarrow^* \beta A, \beta \in (N \cup \Sigma)^*\} \end{aligned}$$

ここで、 $\#$ は入力の終わりを示す記号である。

ステップ 1: $\text{Follow}_1(S) \leftarrow \{\#\}$

ステップ2: $A \in N \setminus \{S\}$ に対して $\text{Follow}_1(A) \leftarrow \emptyset$

(a) $\varepsilon \notin \text{First}_1(\gamma)$ のとき, $\text{First}_1(\gamma)$

(b) $\varepsilon \in \text{First}_1\gamma$ のとき, $\text{First}_1(\gamma) \setminus \{\varepsilon\} \cup \text{Follow}_1(B)$

4.2.1 左再帰性

$$\begin{aligned} \langle \text{proc_decl} \rangle &::= \text{'procedure' } \langle \text{IDENT} \rangle \text{'(' } \langle \text{id_list} \rangle \text{')' ':' } \langle \text{inblock} \rangle \\ \langle \text{id_list} \rangle &::= \langle \text{id_list} \rangle \text{' , ' } \langle \text{IDENT} \rangle \\ &\quad | \text{'IDENT' } \end{aligned}$$
$$\begin{aligned} \langle \text{id_list} \rangle &\Rightarrow_{lm} \langle \text{id_list} \rangle', 'IDENT' \\ &\Rightarrow_{lm} \langle \text{id_list} \rangle', 'IDENT', 'IDENT' \\ &\Rightarrow_{lm} \langle \text{id_list} \rangle', 'IDENT', 'IDENT', 'IDENT' \\ &\vdots \end{aligned}$$

左再帰性除去

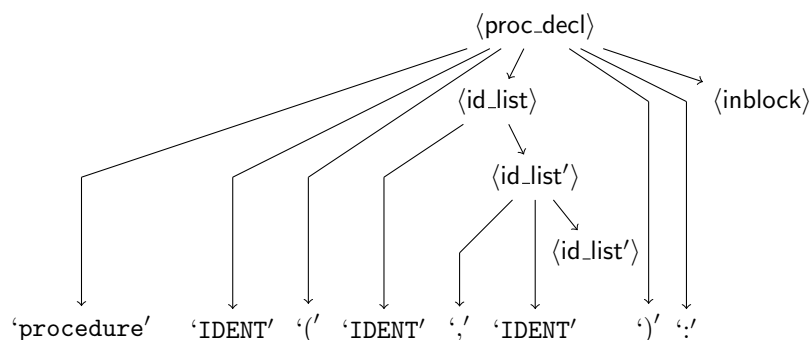
$$\begin{array}{lcl} A & \rightarrow & \beta A' \\ A' & \rightarrow & \alpha A' \\ A' & \rightarrow & \varepsilon \end{array}$$

A' は新たな非終端記号である。

これに従って $\langle id_list \rangle$ に対して左再帰性を除去すると以下のようなになる。

$$\begin{aligned}\langle id_list \rangle &::= 'IDENT' \langle id_list' \rangle \\ \langle id_list' \rangle &::= ', 'IDENT' \langle id_list' \rangle \\ &\quad | \varepsilon\end{aligned}$$

この場合の解析木は以下のようなになる。



例 4.3. 加算 $+$ と乗算 $*$ を持つ式に対して、左再帰性をもつ生成規則と E からの導出が等しい左再帰性を除去した生成規則は以下のようなになる。

$\begin{aligned}E &\rightarrow E + T \mid T \\ T &\rightarrow T * F \mid F \\ F &\rightarrow (E) \mid 'IDENT'\end{aligned}$	$\begin{aligned}E &\rightarrow TE' \\ E' &\rightarrow +TE' \mid \varepsilon \\ T &\rightarrow FT' \\ T' &\rightarrow *FT' \mid \varepsilon \\ F &\rightarrow (E) \mid 'IDENT'\end{aligned}$
(a) 左再帰性を持つ生成規則	(b) 左再帰性を除去した生成規則

4.2.2 左括り出し

以下の BNF 定義において

$$\begin{aligned}\langle conditional \rangle &::= 'if' \langle expression \rangle 'then' \langle statement \rangle 'else' \langle statement \rangle 'fi' \\ &\quad | 'if' \langle expression \rangle 'then' \langle statement \rangle 'fi'\end{aligned}$$

予約語 $'if'$ を見ただけではどちらの規則を適用すればよいかわからない。さらに新たな非終端記号 $\langle conditional' \rangle$ を導入し、

$$\begin{aligned}\langle conditional \rangle &::= 'if' \langle expression \rangle 'then' \langle statement \rangle \langle conditional' \rangle \\ \langle conditional' \rangle &::= 'else' \langle statement \rangle 'fi' \\ &\quad | 'fi'\end{aligned}$$

とすれば、 $\langle conditional \rangle$ の規則は単一になり、 $\langle conditional' \rangle$ についても先読みトークンが $'else'$ か $'fi'$ かで決定的に規則を選択できる。

同じ左辺を持つ生成規則において、右辺の共通な左端をまとめることを**左括り出し** (left-factoring) という。一般には $A \rightarrow \alpha\beta_1 \mid \alpha\beta_2$ に対して、新たな非終端記号 A' を導入して $A \rightarrow \alpha A'$, $A' \rightarrow \beta_1 \mid \beta_2$ に変換することである。